

PROJETO INTEGRADOR

Casa de Apostas Provably Fair sobre a Rede BSD

Laboratório de Hardware + Laboratório de Software

Plano de Projeto e Guia Pedagógico de Execução

Prof. Alan Mateus — Técnico em Informática / Desenvolvimento de Sistemas (EMTI)

Barbacena, Minas Gerais — 2026

Sumário

1. Visão Geral e Objetivo Pedagógico
2. Regra de Ouro do Projeto: Execução Prática ou Documentação (POP)
3. Alinhamento com as Ementas das Disciplinas
4. Divisão da Equipe em 4 Funções Corporativas
5. Cronograma de Execução (4 Sprints)
6. Padrão Corporativo de Git, GitHub e Terminal
7. O MVP do Cassino: Jogo e Protocolo Commit-Reveal
8. A Prova Matemática: Por Que a Casa Sempre Ganha
9. Conexão com a Moeda BSD e a Rede Blockchain
10. Estrutura Obrigatória do Repositório GitHub
11. Roteiro para a Apresentação Final
12. Matriz e Rubrica de Avaliação Individual

1. Visão Geral e Objetivo Pedagógico

Vocês vão desenvolver um Sistema de Casa de Apostas Provavelmente Justo (Provably Fair), rodando sobre a moeda BytesDCoin (BSD) e sua sub-unidade PRIME. O projeto une, em um único trabalho, infraestrutura física (Laboratório de Hardware), sistemas operacionais e automação (Laboratório de Software), engenharia de blockchain, matemática probabilística e trabalho colaborativo em equipe via Git/GitHub.

1.1. Os três desafios do projeto

- Engenharia e infraestrutura: montar/testar a máquina, medir consumo elétrico sob estresse, configurar Linux, operar por linha de comando e versionar o código como uma equipe de empresa.
- Desenvolvimento do MVP: construir uma aplicação web funcional de um jogo simples de apostas (ex.: Dado 1–100), com saldo em PRIMEs e auditoria pública do resultado.
- Prova matemática: rodar uma simulação Monte Carlo com 10.000+ apostas e provar, com números e gráfico, que o valor esperado do apostador é negativo — ou seja, no longo prazo o jogador sempre perde, e a única forma de ele não perder capital é mudando a arquitetura do jogo.

Modelo MVP + Prova de Conceito (PoC)

Vocês não vão construir uma blockchain Layer-1 completa do zero em um semestre — isso é irreal até para equipes profissionais. O objetivo é construir de verdade a parte central (o jogo, a lógica anti-fraude, o bot de simulação) e documentar tecnicamente, com passo a passo real, a parte de infraestrutura que não for possível executar fisicamente. Isso é exatamente o que empresas de tecnologia fazem: um MVP funcional + uma especificação de arquitetura para o que vem depois.

2. Regra de Ouro do Projeto: Execução Prática ou Documentação (POP)

Regra obrigatória

Toda etapa prática que não puder ser executada de verdade por falta de equipamento, permissão de sistema, hardware dedicado ou tempo de laboratório DEVE ser substituída por um Procedimento Operacional Padrão (POP) documentado. Projetos sem execução real E sem essa documentação técnica serão penalizados na rubrica de avaliação.

Um POP completo precisa ter três partes:

- 1. Mapeamento do impedimento: por que aquela etapa não pôde ser executada na prática (explicação técnica real, não “não deu tempo”).

- 2. Roteiro passo a passo completo: os comandos de terminal, esquemas elétricos, arquivos de configuração ou código exatos que seriam usados em um ambiente real de produção.
- 3. Justificativa técnica: o porquê de cada comando, parâmetro ou escolha — não basta copiar um comando, é preciso explicar o que ele faz e por que é necessário.

Exemplos de quando aplicar a regra: não ter multímetro/wattímetro para medir consumo elétrico real; não ter uma GPU dedicada para estresse de mineração; não conseguir compilar uma biblioteca nativa de criptografia pós-quântica; não ter uma segunda máquina para simular múltiplos nós P2P.

3. Alinhamento com as Ementas das Disciplinas

3.1. Laboratório de Hardware

Item da ementa	Como o projeto conecta
Normas de segurança, ESD e aterramento	Checklist de bancada segura antes de qualquer teste de estresse na máquina de validação
Montagem e desmontagem de hardware	Dimensionamento de CPU/GPU capazes de rodar o algoritmo de memória dura Argon2id (usado na validação da rede BSD)
Instalação/desinstalação de drivers	Instalação e otimização de drivers de vídeo (OpenCL/CUDA) para processar hashing sem travar
Equipamentos de medição	Medição de tensão (V), corrente (A) e potência (W) em repouso e sob carga de estresse, com multímetro/wattímetro
Normas de reparo e manutenção	Manutenção preventiva (pasta térmica, ventilação) em máquinas submetidas a carga alta de processamento

3.2. Laboratório de Software

Item da ementa	Como o projeto conecta
Linha de comando, pipes e filtros	Uso da CLI do projeto (grep, jq) para consultar logs e transações
Comandos em lote	Scripts .sh/.bat para inicializar a aplicação e automatizar backup/restore
Criação/execução de máquinas virtuais	Ambiente em VM ou Docker com a stack do projeto pré-configurada
Backup, restore e recuperação a desastres	Cópia de segurança do banco de dados do sistema e simulação de restauração após falha
Configuração de diretivas de segurança	Permissões restritas (chmod 600) nos arquivos de chaves privadas
Correção de falhas no sistema de arquivos	Diagnóstico de travamentos do banco local após desligamento incorreto

4. Divisão da Equipe em 4 Funções Corporativas

O grupo funciona como uma pequena empresa de tecnologia. Cada função tem um dono, mas todos revisam o trabalho de todos via Pull Request.

Função	Foco principal	Entregável individual
1. Dev Frontend & UI	Interface do jogo, exibição de saldo em PRIMEs, painel de auditoria do Commit-Reveal	Aplicação web jogável em src/frontend/
2. Dev Backend & Matemática	Motor Commit-Reveal, bot de simulação Monte Carlo, prova matemática	src/backend/, src/bot/ e docs/MATHEMATICAL_PROOF.md
3. DevOps & SysAdmin	Repositório GitHub, scripts de terminal, backup/restore, permissões do SO	Pasta /scripts, README.md, histórico de Pull Requests
4. Hardware Lead	Testes de estresse, medição elétrica, normas ESD/aterramento	docs/HARDWARE_SETUP.md com dados e checklist

4.1. O que cada função pesquisa, planeja e explica

Dev Frontend & UI

- Como o painel de auditoria mostra ao usuário o Commit Hash e a Semente Revelada para ele conferir sozinho.
- Como exibir saldo em PRIMEs de forma amigável (sem casas decimais confusas na tela).

Dev Backend & Matemática

- Como implementar SHA3-256(conteúdo + endereço + salt) em JavaScript para o Commit-Reveal.
- Como programar o bot que roda milhares de apostas automáticas e registra o saldo a cada rodada.
- Como calcular e apresentar o Valor Esperado ($E[X]$) do jogo escolhido pela equipe.

DevOps & SysAdmin

- Como estruturar branches (main, develop, feature/*) e revisar Pull Requests com pelo menos 1 aprovação.
- Como escrever um script de backup/restore do banco de dados local e testar a recuperação após uma falha simulada.

Hardware Lead

- Como medir tensão, corrente e potência da máquina em repouso e sob carga (ou documentar o POP se não houver equipamento).
- Como calcular teoricamente o consumo (TDP) dos componentes quando a medição real não for possível.

5. Cronograma de Execução (4 Sprints)

Sprint (semana)	Foco	Atividades principais
1 — Setup & Arquitetura	Planejamento e GitHub	Definir as 4 funções; criar repositório com branches main/develop e quadro Kanban; clonar um jogo simples

		(Dado ou Cara-ou-Coroa) como base; preparar bancada e ferramentas de medição
2 — Desenvolvimento Core	Lógica, interface e hardware	Adaptar a interface para PRIMEs; implementar Commit-Reveal; rodar teste de estresse de CPU/GPU; escrever scripts de inicialização
3 — Testes & Automação	Monte Carlo e SO	Executar o bot com 10.000+ apostas e gerar o gráfico de queda de saldo; implementar backup/restore e permissões; conectar a aba de auditoria
4 — Entrega & Defesa	Revisão e apresentação	Fechar Pull Requests pendentes; finalizar README.md, MATHEMATICAL_PROOF.md e HARDWARE_SETUP.md; ensaiar a demonstração ao vivo

6. Padrão Corporativo de Git, GitHub e Terminal

```
[ branch: main ]      <- código estável, só chega aqui via Pull Request aprovado
|
[ branch: develop ]   <- integração contínua da equipe
|
+---+-----+
|                                     |
[ feature/commit-reveal-logic ]   [ feature/ui-casino-layout ]
```

6.1. Regras obrigatórias

- Nenhum commit direto em main ou develop — tudo passa por uma branch feature/nome-da-tarefa.
- Conventional Commits: feat (funcionalidade nova), fix (correção), docs (documentação), test (testes/simulações), style (formatação).
- Todo Pull Request precisa de pelo menos 1 aprovação de um colega antes do merge (Code Review).
- Gestão de tarefas no GitHub Projects, com colunas Backlog / In Progress / Review / Done.

7. O MVP do Cassino: Jogo e Protocolo Commit-Reveal

7.1. Jogo recomendado — Dado 1 a 100 (mais fácil de implementar)

Regra: o apostador escolhe um número e aposta se o resultado sorteado será maior ou menor que ele. A probabilidade é direta (número escolhido ÷ 100), o que facilita tanto a implementação quanto a prova matemática. Alternativas mais simples ainda: Cara ou Coroa, ou um jogo de multiplicador (“foguetinho”). A equipe pode clonar um repositório open-source (licença MIT/Apache) de um jogo desse tipo e adaptar a lógica para PRIMEs e Commit-Reveal.

7.2. Fluxo completo do protocolo

```
ETAPA 1 — COMMIT (antes da aposta)
Servidor gera semente secreta S e um salt único
Servidor calcula: Hash = SHA3-256(S + Salt)
Servidor envia o Hash ao jogador ANTES de receber a aposta

ETAPA 2 — APOSTA
```

```
Jogador escolhe o palpite e o valor em PRIMES
Jogador envia a aposta

ETAPA 3 – REVEAL & VALIDAÇÃO
Servidor calcula o resultado a partir de S e Salt
Servidor revela S e Salt ao jogador
Jogador recalcula SHA3-256(S + Salt) e confere que bate com o Hash da Etapa 1
-> Se bater, o resultado é comprovadamente justo (não foi trocado depois da aposta)
```

8. A Prova Matemática: Por Que a Casa Sempre Ganha

8.1. Valor Esperado ($E[X]$)

Para um jogo com 49% de chance de ganhar $1\times$ a aposta e 51% de chance de perder (margem da casa de 2%):

```
 $E[X] = (0,49 \times +1) + (0,51 \times -1) = -0,02$  PRIME por PRIME apostado
```

A cada 100 PRIMES apostados, o jogador perde em média 2 PRIMES. Repetido milhares de vezes, isso é a Teoria da Ruína do Apostador.

8.2. O experimento: bot de simulação Monte Carlo

O bot deve rodar no terminal, fazer 10.000 (ou mais) apostas automáticas contra o motor do jogo e registrar o saldo a cada intervalo, exportando um gráfico da queda do saldo ao longo do tempo. Exemplo de saída esperada:

```
=====
SIMULADOR MONTE CARLO - CASINO BSD (PROVABLY FAIR)
=====
[CONFIG] Apostas: 10.000 | Aposta base: 100 PRIMES | Margem casa: 2,0%
- Aposta #1.000 | Saldo: 982.100 PRIMES (perda: -1,79%)
- Aposta #5.000 | Saldo: 899.400 PRIMES (perda: -10,06%)
- Aposta #10.000 | Saldo: 798.200 PRIMES (perda: -20,18%)
=====
[RESULTADO] Lucro da casa: 201.800 PRIMES (20,18% de margem acumulada)
[CONCLUSAO] A Teoria da Ruína do Apostador foi confirmada.
=====
```

8.3. A pergunta central do projeto: existe uma casa 100% justa?

A equipe deve responder, no relatório MATHEMATICAL_PROOF.md, à seguinte pergunta usando qualquer ferramenta matemática necessária:

Pergunta central

É possível criar uma casa de apostas onde o jogador nunca perca seu capital principal, mas a casa ainda consiga lucrar?

Caminho de raciocínio esperado:

- No modelo tradicional (transferência direta de saldo entre jogador e casa), é matematicamente impossível os dois lados ganharem ao mesmo tempo — o jogo é de soma negativa.
- Modelo alternativo (No-Loss Pool): os PRIMES apostados ficam travados em um contrato de custódia (Smart Escrow) em vez de serem transferidos à casa. Esse montante gera rendimento ao participar da

validação de blocos da rede BSD. O prêmio do vencedor vem apenas desse rendimento, e todo mundo recupera 100% do valor que apostou.

- Conclusão esperada: só existe “aposta sem perda” quando o jogo deixa de ser um cassino tradicional e passa a ser um pool de investimento coletivo — a casa nunca ganha às custas direta do jogador que perdeu.

9. Conexão com a Moeda BSD e a Rede Blockchain

O sistema de apostas não usa a moeda BSD apenas como “dinheiro do jogo” — ele também contribui com a rede, exatamente como um minerador contribui com o Bitcoin:

- Roda como nó (mesmo que simulado/documentado): guarda uma cópia do estado da blockchain e retransmite blocos e transações.
- Processa validação em segundo plano: nos períodos de menor movimento, o hardware da aplicação pode processar o Canal Gaming (Argon2id) da rede BSD e receber recompensa de bloco por isso.
- Usa Gas Tanks: o operador do cassino pode patrocinar as micro-taxas dos jogadores, do mesmo jeito que um professor patrocina as taxas dos alunos numa prova.
- Auditoria pública: qualquer apostador pode inspecionar os hashes do Commit-Reveal gravados na blockchain e confirmar que o jogo não foi manipulado.

10. Estrutura Obrigatória do Repositório GitHub

```
.github/workflows/      <- CI/CD (testar scripts automaticamente)
docs/
├── ARCHITECTURE.md      <- Como o sistema se conecta à rede BSD
├── MATHEMATICAL_PROOF.md <- Gráficos Monte Carlo + prova do Valor Esperado
└── HARDWARE_SETUP.md   <- Medição elétrica, drivers, SO e POPs de contingência
src/
├── frontend/           <- Interface do jogo (HTML/CSS/JS)
├── backend/             <- Motor Commit-Reveal e regras do jogo
└── bot/                 <- Bot de simulação Monte Carlo
scripts/
├── start_app.sh         <- Inicializa a aplicação
├── setup_node.sh        <- Configura o ambiente no Linux
└── backup_leveladb.sh   <- Backup/restore do banco de dados local
README.md               <- Guia principal: instalação, uso e resumo dos
resultados
```

11. Roteiro para a Apresentação Final

- 1. Capa e introdução: visão geral do projeto (hardware + software + blockchain + apostas).
- 2. Arquitetura em camadas: hardware → sistema operacional/CLI → protocolo BSD → aplicação de apostas.
- 3. Laboratório de Hardware: normas ESD, aterramento, medições elétricas (reais ou POP teórico).
- 4. Laboratório de Software: scripts, backup/restore, permissões, virtualização.
- 5. A moeda PRIME e o protocolo Commit-Reveal: como funciona a segurança pós-quântica e o anti-fraude.
- 6. Demonstração ao vivo: rodar a aplicação, fazer uma aposta, mostrar a auditoria, rodar o bot Monte Carlo.
- 7. Conclusão: a prova matemática da ruína do apostador e a resposta à pergunta “existe aposta sem perda?”.

12. Matriz e Rubrica de Avaliação Individual

Função	Critério de avaliação	Evidência no GitHub
Dev Frontend	Qualidade da interface e do painel de auditoria	Branches feature/ui-*, arquivos em src/frontend/
Dev Backend & Math	Correção do Commit-Reveal, qualidade do bot e da prova matemática	src/backend/, src/bot/, docs/MATHEMATICAL_PROOF.md
DevOps & SysAdmin	Organização do GitHub, scripts funcionando, permissões corretas	Pasta /scripts, README.md, histórico de Pull Requests
Hardware Lead	Precisão das medições (ou qualidade do POP), normas ESD/aterramento	docs/HARDWARE_SETUP.md com dados/fotos/POP

Penalização: qualquer etapa não executada na prática e sem o POP correspondente (Seção 2) reduz a nota daquela função específica, mesmo que o restante do trabalho esteja completo.